Benchmarking LLM-Augmented Web Vulnerability Detection

Juho Choi
University of Waterloo
j257choi@uwaterloo.ca

Raghav Mehta
University of Waterloo
r5mehta@uwaterloo.ca

Advait Sangle
University of Waterloo
asangle@uwaterloo.ca

Louis Song
University of Waterloo
m62song@uwaterloo.ca

Tengyi Xu
University of Waterloo
t48xu@uwaterloo.ca

1 Introduction and Motivation

Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) are the established automated methods for detecting vulnerabilities in web applications, operating on source code and on runtime behavior respectively. Both produce high false-positive rates, so their findings require manual triage before they are actionable.

Large language models (LLMs) have recently been applied to the same task, and they shift this trade-off rather than removing it. Recent comparisons report that LLMs detect a far higher fraction of vulnerabilities than SAST tools, in some settings 90 to 100%, but at a much higher false-positive rate, and that combining the two recovers some of each approach's weaknesses [1]. This leaves a practical security engineering problem for anyone assembling a detection pipeline today: given SAST, DAST, plain LLMs, scanner-assisted LLMs, and agentic configurations, which approaches and combinations are worth using, and at what cost in false positives, compute, and latency?

This project investigates that problem empirically. We assemble the main detection approaches together with a range of LLM strategies (prompting variants, scanner-assisted triage, and agentic configurations that operate the scanners) and evaluate them against each other on the same vulnerable applications, under one scoring procedure with known ground truth, to give a clear, realistic account of the accuracy, false-positive, and cost trade-offs across these options.

2 Related Work

2.1 SAST/DAST/LLM Trade-offs and Scanner-Assisted Hybrids

SAST and DAST are the two established families of automated web-application vulnerability detection, and they rest on opposite assumptions. A SAST tool analyzes source code without running it, tracing data and control flow over a parsed representation of the program and matching it against vulnerability patterns; it needs source access and a language-specific front end, sees every code path in principle, but over-approximates, flagging paths whose exploitability it cannot confirm and so producing false-positive rates that routinely exceed 50%. A DAST tool instead probes a running application from the outside, sending crafted inputs and inspecting the responses; it needs a deployed app rather than source, is largely language-agnostic, and reports only behavior that actually manifests, but it under-approximates, missing whatever its inputs never reach and unable to localize a finding back to a line of code. LLM-based detection sits at the opposite end of the SAST trade-off from the scanner: models detect 90 to 100% of vulnerabilities in some settings but at a far higher false-positive cost, and combining a model with a scanner recovers part of each weakness [1].

Prior work therefore pairs an LLM with a scanner rather than choosing between them, and each such pairing improves on the tool alone. LLM4SA feeds each SAST warning and its surrounding code to an LLM that judges whether the warning is real, reporting 81% precision across the Juliet suite and real C/C++ projects and covering the broad range of CWE classes those static warnings span [2]. IRIS combines an LLM with CodeQL for whole-repository taint analysis on Java, and on CWE-Bench-Java detects 55 vulnerabilities against CodeQL's 27 while lowering the false discovery rate, targeting the injection-style classes that taint tracking captures [3]. ZeroFalse keeps the same warning-triage pattern but enriches each warning with flow-sensitive traces and CWE-specific context, so the model reasons over the actual tainted path rather than an isolated alert [4]. How close such triage can come to replacing the scanner is shown by a comparison of thirty LLMs against Semgrep on the Java OWASP Benchmark v1.2, where Semgrep scores F1 0.66 and the best model reaches 0.88, though it treats the model as a replacement for the scanner rather than a complement [5]. A static baseline should therefore include not only a scanner and an unaided model but this kind of pairing.

Two results narrow the design on the orchestration and dynamic sides. Sifting the Noise evaluates three agent frameworks (Aider, OpenHands, and SWE-agent) that drive a scanner and re-examine its warnings for SAST false-positive filtering on the Java OWASP Benchmark, cutting an initial 92% false-positive rate to as low as 6%, but finds the gain over plain prompting strongly backbone-dependent: stronger models benefit substantially from agentic scaffolding while weaker ones improve little or inconsistently [6]. On the dynamic side, a constrained LLM sits atop conventional web scanners as a post-scan reasoner with no direct tool calls, reading scan output and re-judging it, which raised micro-averaged F1 from 0.58 to 0.82 and cut the false-alarm rate from 23.4% to 7.2% across WebGoat, DVWA, and bWAPP, spanning Java and PHP web targets and classic classes such as injection and cross-site scripting; its authors likewise found the tightly scoped assistant beat an autonomous agent [7]. Both point

the same way: the gain from added orchestration is real but conditional.

2.2 Structural and Retrieval-Based Context Augmentation

Beyond scanner pairing, a separate line of work varies how the code is presented to the model. Structural augmentation feeds a parsed representation rather than flat tokens: VulTrLM decomposes a function's abstract syntax tree into annotated subtrees to capture multi-statement execution sequences that a flat sequence misses [8], and VulACLLM fuses AST and control-flow-graph features directly, reporting recall 0.73 and F1 0.725 at function granularity [9]. Retrieval-augmented prompting grounds the model in an external knowledge base instead: Vul-RAG retrieves vulnerability-level knowledge, both cause and fix pattern, where earlier graph- and pretrained-model methods encode only AST or CFG structure [10]. A third strand targets input scope and cost: LLMxCPG slices a combined AST, CFG, and program-dependence-graph representation down to the vulnerability-relevant statements, cutting code size by 68 to 91% while preserving context and enabling multi-function analysis [11], and a study of ten open models finds tokenized input length has an inconsistent but sometimes significant effect on accuracy, with GPT-4, Mistral, and Mixtral notably robust and others degrading [12].

2.3 Prompting Strategy

Prompting strategy forms a further axis. Chain-of-thought prompting has the model summarize a function's behavior, reason about candidate errors, then reach a verdict, improving precision consistently while its effect on recall is more variable [13]; the gain is not universal, since GPT-3.5 and GPT-4 both degrade under chain-of-thought on the harder PrimeVul C/C++ pair set [14]. Vulnerability-semantics-guided prompting sharpens the idea by steering reasoning toward the specific behaviors responsible for vulnerable execution rather than generic step-by-step thinking [15]. A simpler precedent chains two prompts, having the model first describe a snippet's intent and then judge its vulnerability so the second stage can exploit the first's context [16], and a top-down summarization pass is a complementary decomposition applied before such a chain rather than within it. Few-shot prompting, which places labeled examples before the target snippet, is reported alongside chain-of-thought as comparably effective but less studied for this task [13].

2.4 Multi-Agent Role Decomposition

The last line decomposes the task across multiple agents. GPTLens splits detection into an auditor agent that proposes candidate vulnerabilities and a critic agent that scores them, with the critic's score final, evaluated on Solidity smart contracts [17]. VulTrial recasts this as a courtroom with four roles, a security researcher, a code author, a moderator, and a review board, where the review board rather than a lone critic decides [18]. VulAgent drops role-play for a hypothesis-validation loop that forms a hypothesis, derives a trigger path, and verifies it against context, reporting roughly a 36% reduction in false-positive rate and higher pairwise correctness than both GPTLens and VulTrial on the C/C++ PrimeVul and SVEN sets [19]. iAudit splits the task across detector, reasoner, critic, and ranker roles combined with fine-tuning [20], while AutoPatch adds a verifier agent that builds an explicit verification prompt against a retrieved reference vulnerability, a structure closer to a scanner-then-verifier arrangement than to freeform auditing [21].

Taken together, this work shows LLM-based detection varying along several largely independent dimensions, structural context, external grounding, input scope, reasoning strategy, and role decomposition, each with demonstrated but conditional gains over an unaided model, and spread across different languages and vulnerability classes rather than one shared target. No existing study evaluates these dimensions against one another, or against SAST and DAST baselines, under a single scoring procedure with shared cost and latency accounting. Our evaluation is built to close that gap: it places scanner-assisted pairings at the core, treats agentic and multi-agent configurations as conditions to test against that core rather than assumed winners, and compares static and dynamic approaches side by side, which prior work has not done.

3 Proposed Work

We evaluate the main detection approaches and a set of LLM strategies against each other under one scoring procedure. The starting configurations are:

- **A1: Semgrep only** (SAST baseline).
- **A2: OWASP ZAP only** (DAST baseline).
- **A3: LLM only** (unaided model given the code or running app).
- **B1: LLM + Semgrep output** (model receives SAST findings).
- **B2: LLM + ZAP output** (model receives DAST findings).
- **C1: Multi-agent role decomposition** (specialized agent roles, such as a scanner paired with a separate verifier, passing findings between them; Section 2.4) [17–19].
- **C2: AST-augmented context** (the model is given the abstract syntax tree in place of raw text; Section 2.2) [8].
- **C3: Graph-augmented context** (a control- or data-flow graph instead of raw text; Section 2.2) [9].
- **C4: Retrieval-augmented prompting** (CWE entries closest to the snippet retrieved and embedded in the prompt; Section 2.2) [10].
- **C5: Input scoping** (code sliced down to the lines around the risky site; Section 2.2) [11].
- **C6: Top-down summarization** (code summarized from module to statement before any judgment; Section 2.3).

- **C7: Few-shot prompting** (labeled examples placed before the target snippet; Section 2.3) [13].
- **C8: Chain-of-thought prompting** (a step-by-step reasoning pass before the verdict; Section 2.3) [13, 15].
- **C9: Three-prompt chain** (summarize, then flag candidate issues, then examine the flagged parts in depth; Section 2.3) [16].

C1 to C9 are candidates for more complicated LLM-based techniques, and only four or five of them will be selected to carry through the full evaluation, so that each reported result stays attributable to a single technique and the compute and token budget stays realistic. The A and B configurations, by contrast, are all run.

We treat these as starting points and experiment with the LLM strategies that matter most in practice: prompt design, how scanner output is presented to the model, how the model can improve the scanner itself by authoring rules, and, as an exploratory extension, composing specialized agent roles (for example a scanning agent and a separate verifier agent) around the scanners. The evaluation runs through a single harness with a pluggable model backend, so each configuration can be driven by a local model or, with an API key, a hosted one, which also lets us compare a local model against a frontier model on identical inputs. Configurations are scored on the OWASP Benchmark, whose labeled test cases give automated ground truth and which is scored identically for SAST and DAST. A set of realistic vulnerable applications (OWASP Juice Shop, WebGoat, DVWA) serves as a secondary, qualitative target to see how the approaches transfer beyond the synthetic benchmark. Each configuration is measured on detection accuracy (precision, recall, F1), false-positive rate, and per-run cost and latency, so the comparison reflects engineering trade-offs rather than detection in isolation. The research questions are framed around those trade-offs:

- **RQ1.** Run alone (A1, A2, A3), how do SAST, DAST, and an unaided LLM compare on detection accuracy against false-positive rate and cost?
- **RQ2.** Does pairing an LLM with a scanner by triaging its output (B1, B2) improve the accuracy versus false-positive trade-off over the scanner alone, and is the added cost justified?
- **RQ3.** What do more advanced AI strategies, including multi-agent orchestration, chain-of-thought prompting, and retrieval-augmented configurations, add to detection accuracy and false-positive control beyond the scanner-assisted pairings in RQ2, and under what conditions does the added complexity justify the cost?

To bound scope, the multi-agent configuration uses a fixed, small set of agent roles, and all scored runs use one designated machine and one model so that configurations remain directly comparable.

4 Plan of Action & Rationale

The plan follows the incremental pipeline from the proposal and reuses its configuration labels, built on a single evaluation harness, llm-vulnbench (https://github.com/advaitsangle/llm-vulnbench). Each phase adds one class of detector on top of the last, so any change in the numbers can be traced to a specific method.

4.1 Phase 1: Baselines and harness

- Stand up llm-vulnbench, with every condition written as an independent class behind one interface, `run(target) -> findings + usage`.
- Run and lock the two scanner baselines: Semgrep for static analysis (A1, SAST) and OWASP ZAP for dynamic analysis (A2, DAST).

Rationale. The scanners are the fixed reference every later condition is measured against, so they are locked first. Building the harness around a single `run` interface is what makes the rest tractable: a model, a prompting style, or a whole configuration can be swapped in without touching the pipeline, which also lets us compare a local model against a hosted frontier model on identical inputs. Normalizing every condition to one schema and one scoring procedure is what makes the numbers comparable at all.

4.2 Phase 2: LLM-assisted SAST and DAST

- B1: the model triages Semgrep's SAST findings to filter false positives.
- B2: the model triages OWASP ZAP's DAST findings in the same evaluator role.

Rationale. This phase isolates the core question of whether pairing a model with a scanner beats the scanner alone, on both the static and dynamic sides, and whether a model can help by cleaning up a tool's output. We treat the gain as an open question rather than an assumption, because prior work shows it is conditional, depending on how scanner and model are combined and on the strength of the underlying model [2, 3, 6, 7], so a technique that helps one pipeline can do little for another.

4.3 Phase 3: LLM-only approaches (next steps)

- A3: the unaided model as the simple case, given the code or the running app.
- On top of A3, the candidate techniques C1 to C9 introduced in the proposed work, from which only four or five will be selected to carry through the full evaluation; the remaining ones are dropped.

Rationale. This is the exploratory frontier of the project, hence still in progress. We carry only a handful of C1 to C9 through rather than all of them, so each result stays attributable to a single technique and the compute and token budget stays realistic, which matters when cost is one of the

things we measure. Given the early results (Section 5), these techniques are judged first on whether they let the model reject a finding, since that is where every current condition fails, and not on F1 alone.

4.4 Phase 4: Evaluation and scorecards (next steps)

- Run the condition ladder against the OWASP Benchmark (Java), targeting the full 2,740-case set or as large a subset as our compute and API budget allows.
- Transfer the strongest configurations to the realistic applications used as secondary, qualitative targets, OWASP Juice Shop, WebGoat, and DVWA, with the held-out Python benchmark as a further check.
- Swap the local model for a hosted frontier one on identical inputs, to test whether the rejection failures seen so far are a capability limit or a design one.
- Report precision, recall, F1, and false-positive rate against latency and token cost.

Rationale. Development uses small seeded samples to keep API cost and latency down, but the reported results need as much of the benchmark as we can afford in order to limit sampling bias, and the realistic apps test whether a result survives outside the synthetic benchmark. Accuracy and cost are reported together on purpose: a configuration that lifts F1 by a few points while multiplying latency or token spend is a different engineering proposition from one that does it for free, and our research questions are about that trade-off, not detection in isolation.

5 Progress and Preliminary Results

At this midterm stage we have completed Phases 1 and 2 and begun Phase 3. The harness is built and running end to end; both scanner baselines (A1, A2), the unaided model (A3), the scanner-assisted conditions (B1, B2), and a first multi-agent scaffold (C1) all run against ground truth, and we have our first same-slice comparison across six conditions (Table 1). What remains is building out the remaining C2 to C9 techniques and scaling to the full benchmark.

Each scored cell pins its git SHA, prompt hash, and a clean working tree, and every run is checkpointed to disk with its artifacts, so a result can be reproduced exactly or resumed after interruption. Evaluation slices are drawn from the benchmark by stratified sampling under a fixed seed and stored as named subsets, which lets every condition score the same cases under the same pinned prompt. Alongside precision, recall, F1, and false-positive rate, the scorecard now reports Youden's J ($J = R - \text{FPR}$), which we added because F1 alone cannot distinguish a real detector from one that simply flags everything.

Our first same-slice run exercises this. It scores six conditions on one reachable subset, `reachable20`, under a single pinned prompt and one local model (qwen2.5-coder:14b). The slice is 20 cases stratified five apiece across four Top-25 CWE categories, SQL injection (CWE-89), cross-site scripting (79), path traversal (22), and command injection (78), split 12 real to 8 safe, and restricted to the reachable subset so the automatic false-negative cap does not distort the comparison.

Every LLM condition scores $J \leq 0$, while both unaided scanners score $J > 0$, and F1 ranks the matrix in almost the reverse order, with the multi-agent C1 highest at 0.750 and ZAP lowest at 0.500. The gap between the two metrics is the finding: on F1 the LLM appears to win, but on J it has no discriminative power. The two conditions that read raw code without a scanner pre-filter, A3 and C1, flagged all 8 safe cases (TN = 0, FPR 1.0), and C1 flagged 20 of 20 files, so its J of exactly 0.000 is the score a detector earns by calling everything vulnerable. F1 hides that, reading C1's 0.750 as a clear win over Semgrep's 0.640.

The same behavior appears in both scaffolds that let the model reject findings, and in neither does it. C1's verifier accepted all 20 of its candidates and rejected none, spending 2.6 times A3's runtime to add two true positives and remove no false positives. B1 was handed 13 findings by Semgrep (8 true, 5 false) and returned 12 (7 true, 5 false), keeping every false positive and dropping one true positive, which moved J from Semgrep's $+0.042$ to -0.042; its three true negatives are inherited from the scanner, not earned. B2 shows the sharpest version: ZAP handed it 169 alerts covering 4 true positives and zero false positives, and the model rejected none while adding 8 cases (3 real, 5 safe), taking FPR from 0.00 to 0.625 and J from $+0.333$ to -0.042. B1 and B2 return the identical confusion matrix from two different scanners, so on this slice the scanner input barely moves the model's verdict.

6 Tasks and Timeline

The project runs from June 19 to the August 14 final report, with the midterm report due July 17 and the project presentation on July 30.

- **Week 1 (Jun 19 to 26):** Finalize scope, metrics, and ground truth. Stand up the environment (Docker, OWASP Benchmark, Juice Shop, Semgrep, ZAP) and fix the configuration set.
- **Week 2 (Jun 27 to Jul 3):** Run and validate the baselines (A1, A2, A3); build the scoring harness against ground truth.
- **Week 3 (Jul 4 to 10):** Build the LLM harness and the scanner-assisted configurations (B1, B2).
- **Week 4 (Jul 11 to 17):** Run first combined benchmarks. *Midterm report due July 17.*
- **Week 5 (Jul 18 to 24):** Full configuration runs on OWASP Benchmark (Java), plus the held-out Python Benchmark.
- **Week 6 (Jul 25 to 30):** Analysis and figures. *Project presentation July 30.*
- **Weeks 7 to 8 (Jul 31 to Aug 14):** Qualitative runs on the realistic apps and the agentic demo; final analysis and writeup. *Final report due August 14.*

Cond		TP	FP	FN	TN	P	R	F1	FPR	J
A2	ZAP (DAST)	4	0	8	8	1.000	0.333	0.500	0.00	+0.333
A1	Semgrep (SAST)	8	5	4	3	0.615	0.667	0.640	0.625	+0.042
C1	multi-agent	12	8	0	0	0.600	1.000	0.750	1.00	0.000
B1	LLM triages Semgrep	7	5	5	3	0.583	0.583	0.583	0.625	−0.042
B2	LLM triages ZAP	7	5	5	3	0.583	0.583	0.583	0.625	−0.042
A3	LLM only, flat	10	8	2	0	0.556	0.833	0.667	1.00	−0.167

Table 1: Six conditions on the `reachable20` slice under one pinned prompt and one local model (qwen2.5-coder:14b).

7 Work Delegation

- **Juho Choi:** Baseline scanners. Configures and runs Semgrep (SAST) and OWASP ZAP (DAST), producing baselines A1 and A2.
- **Raghav Mehta:** Evaluation methodology and scoring. Defines the configuration set and metrics, implements the ground-truth scoring scripts, and runs the comparative analysis.
- **Advait Sangle:** LLM and agentic harness. Builds the harness that supplies scanner output to the model, implements B1 + B2, and the multi-agent configuration C1, and the orchestration logic.
- **Louis Song:** LLM-only baseline and reporting. Implements the unaided-model configuration A3, and assembles the figures, tables, written report, and presentation.
- **Tengyi Xu:** Datasets and infrastructure. Containerized environment, target applications, reproducible run configuration, and ground-truth label management.

References

[1] X. Zhou, D.-M. Tran, T. Le-Cong, T. Zhang, I. C. Irsan, J. Sumarlin, B. Le, and D. Lo, "Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection," 2024. arXiv:2407.16235.

[2] C. Wen, Y. Cai, B. Zhang, J. Su, Z. Xu, D. Liu, S. Qin, Z. Ming, and T. Cong, "Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We?," *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 7, 2024.

[3] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in *ICLR*, 2025. arXiv:2405.17238.

[4] M. Iranmanesh, S. Moradi Sabet, S. Marefat, A. Javidi Ghasr, A. Wilson, I. Sharafaldin, and M. A. Tayebi, "ZeroFalse: Improving Precision in Static Analysis with LLMs," 2025. arXiv:2510.02534.

[5] D. Koterba, M. Łukaczyk, and W. Książek, "Leveraging Large Language Models for Advanced Static Code Analysis: Assessing the Feasibility of AI Superseding Traditional Vulnerability Detection Methods," *Information and Software Technology*, 2026. doi:10.1016/j.infsof.2026.108213.

[6] Y. Xiong and T. Zhang, "Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering," 2026. arXiv:2601.22952.

[7] V. S. Gaikwad, M. Kulkarni, S. Deshmukh, J. Mehta, and P. Akolkar, "Penetration Testing for Websites using LLM," Research Square preprint, 2026. doi:10.21203/rs.3.rs-9937475/v1.

[8] S. Zhang, Q. Wang, Q. Liu, E. Luo, T. Peng, et al., "VulTrLM: LLM-Assisted Vulnerability Detection via AST Decomposition and Comment Enhancement," *Empirical Software Engineering*, vol. 31, art. 10, 2026.

[9] J. Liu, G. Lin, H. Mei, F. Yang, and Y. Tai, "Enhancing Vulnerability Detection Efficiency: An Exploration of Light-weight LLMs with Hybrid Code Features," *Journal of Information Security and Applications*, vol. 88, art. 103925, 2025. doi:10.1016/j.jisa.2024.103925.

[10] X. Du et al., "Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG," 2024. arXiv:2406.11147.

[11] A. Lekssays et al., "LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models," in *USENIX Security*, 2025. arXiv:2507.16585.

[12] J. Lin and D. Mohaisen, "Evaluating Large Language Models in Vulnerability Detection Under Variable Context Windows," 2025. arXiv:2502.00064.

[13] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights," 2025. arXiv:2502.07049.

[14] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability Detection with Code Language Models: How Far Are We?," 2024. arXiv:2403.18624.

[15] Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai, "Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities," 2024. arXiv:2402.17230.

[16] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-Enhanced Software Vulnerability Detection Using ChatGPT," 2023. arXiv:2308.12697.

[17] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu, "Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives," 2023. arXiv:2310.01152.

[18] R. Widyasari, M. Weyssow, I. C. Irsan, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang, and D. Lo, "Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection using LLM-Based Agents," 2025. arXiv:2505.10961.

[19] Z. Wang et al., "VulAgent: Hypothesis-Validation based Multi-Agent Vulnerability Detection," 2025. arXiv:2509.11523.

[20] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, "Combining Fine-Tuning and LLM-Based Agents for Intuitive Smart Contract Auditing with Justifications," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1742–1754. doi:10.1109/ICSE55347.2025.00027.

[21] M. Seo, W. Choi, M. You, and S. Shin, "AutoPatch: Multi-Agent Framework for Patching Real-World CVE Vulnerabilities," 2025. arXiv:2505.04195.

[22] OWASP Foundation, "OWASP Benchmark Project." https://owasp.org/www-project-benchmark/

[23] OWASP Foundation, "OWASP Juice Shop." https://owasp.org/www-project-juice-shop/